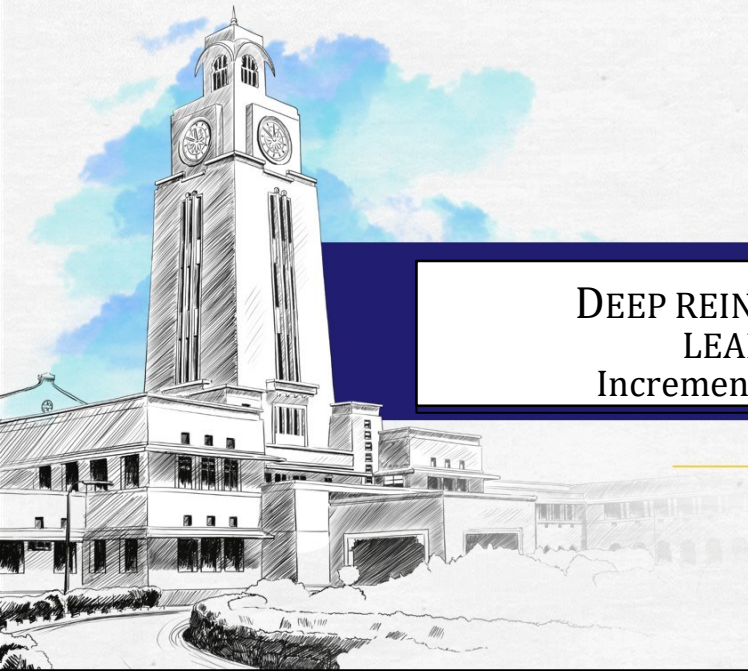




BITS Pilani
Pilani | Dubai | Goa | Hyderabad

DEEP REINFORCEMENT LEARNING Incremental Example

DRL Team
BITS Pilani WILP

The instructors is gratefully acknowledging the
authors who made their course materials
freely available online.

This deck is designed by BITS WILP faculty team of
course Deep Reinforcement Learning

TABLE OF CONTENTS

- 1 [MDP](#)
- 2 [DP](#)
- 3 [MC \(to be updated\)](#)
- 4 [TD \(to be updated\)](#)



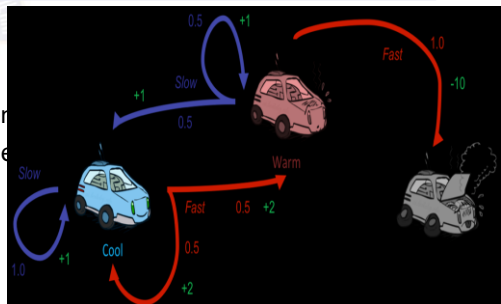
Problem Definition

Race Car Laps

A race car completes laps while managing engine temperature and speed decisions. Here each step is a segment of a lap in the track where the car learns to trade off lap time against overheating risk. The more the lap the car completes its performance gets better. Car shuts down only when its engine gets overheated (optional terminal state).

Key Characteristics:

- The agent must act under **uncertainty** — percepts are local, and the agent must reason about hidden
- Assume there is no fuel constraints or No. of Laps to complete
- At every time step the next potential engine temperature depends on the current engine state not the past history



MARKOV DECISION PROCESS FORMALIZATION (Simplified)

1 States (S)

- $S = (\text{Cool}(\mathbf{C}), \text{Warm}(\mathbf{W}), \text{Overheated}(\mathbf{O}))$ #Engine Temperature
- Finite State MDP

2 Actions (A)

- $A = \{\text{Go Slow}(\mathbf{S}), \text{Go Fast}(\mathbf{F})\}$

Rewards (R)

- $R(s,a,s') = \begin{cases} +1 & \text{if } s' \text{ is reached by going } \textit{Slow} \\ +2 & \text{if } s' \text{ is reached by going } \textit{Fast} \\ -10 & \text{if } s' \text{ is terminal} \end{cases}$

4 State Transition Probabilities (P)

- $P(s'|s,a) = \begin{cases} 0.5 & \text{if action lead to different non-terminal state} \\ 1 & \text{otherwise} \end{cases}$

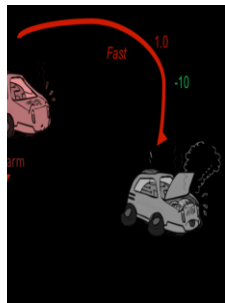


TABLE OF CONTENTS

- 1 [MDP](#)
- 2 [DP](#)
- 3 [MC \(to be updated\)](#)
- 4 [TD \(to be updated\)](#)

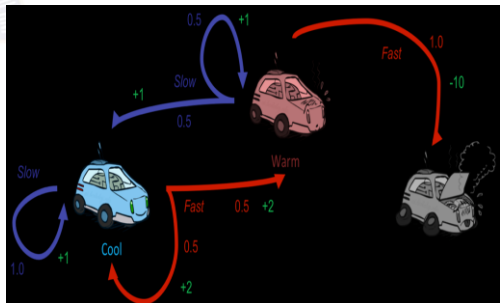


Dynamic Programming

Scenario

In a modified Race car, there is a training track where agent is given a known map and risk factors(dynamics) in advance. Agent knows exactly where it is and its action how actions result in engine state change (KNOWN MODEL).

With full knowledge, the agent can compute optimal policies by trying through all states.



Dynamic Programming

1 Given

- $P(s' | s, a)$, $R(s, a, s')$, $P(s', r | s, a)$

2 To find

- Policy π

3 Solution

- Policy Iteration (PI): Prefer this in a relatively smaller state space where intermediate termination provides explicit interpretable policy
- Value Iteration (VI): Prefer this in a relatively larger state space where fast convergence to optimal policy is obtained at the end

4 Idea

- PI : One Step Look ahead ☐ Evaluate ☐ Improve
- VI : One Step Look ahead ☐ Control

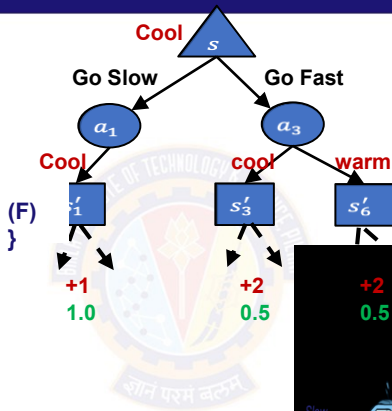
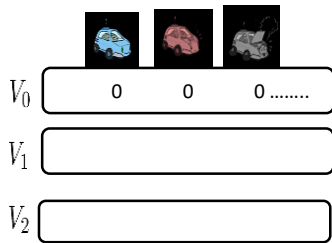
VI can be thought of truncated PI where evaluation is done fully before improvement.

Dynamic Programming Solution



Value Iteration Algorithm

Dynamic Programming – Value Iteration



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}$)

Repeat $\epsilon = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

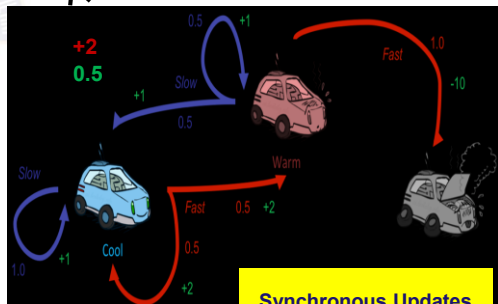
$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

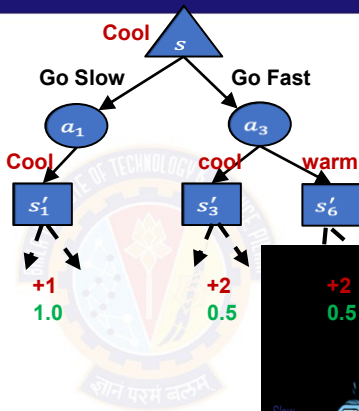
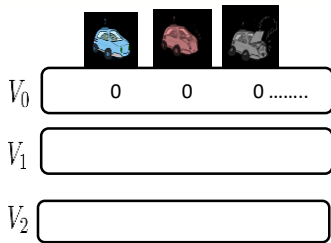
Output a deterministic policy, $\pi \approx \pi_*$, such that

$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$



Synchronous Updates

Dynamic Programming – Value Iteration



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat $\epsilon = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

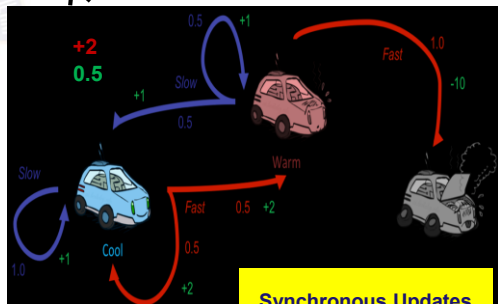
$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that

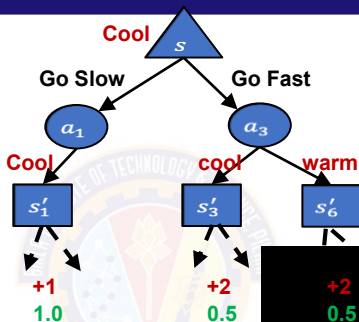
$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$



Synchronous Updates

Dynamic Programming – Value Iteration

			
V_0	0	0	0
V_1	2	1	0
V_2			



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}$)

Repeat $\theta = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that

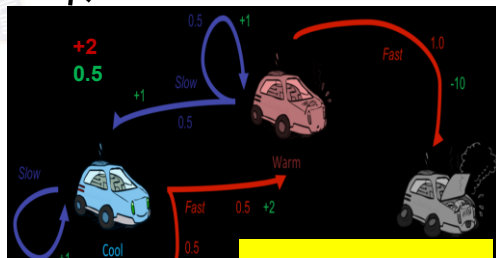
$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

- Below is done only for state : "Cool". Repeat this for all the states "Warm", "Overheated" per iteration

$$V(\text{cool}) = \max \begin{cases} \text{slow: } 1 \\ \text{fast: } 2 \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} 1 * [1 + 0.9(0)] \\ (0.5 * [2 + 0.9(0)]) + (0.5[2 + 0.9(0)]) \end{cases}$$

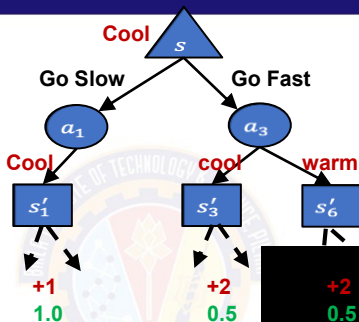
$$V(\text{cool}) = \max \begin{cases} P(c|c,s)[r(c,s,c) + 0.9(v(c))] \\ (P(c|c,f)[r(c,f,c) + 0.9(v(c))]) + (P(w|c,f)[r(c,f,w) + 0.9(v(w))]) \end{cases}$$



Synchronous Updates

Dynamic Programming – Value Iteration

			
V_0	0	0	0
V_1	2	1	0
V_2	3.35	2.35	0



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat $\epsilon = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that

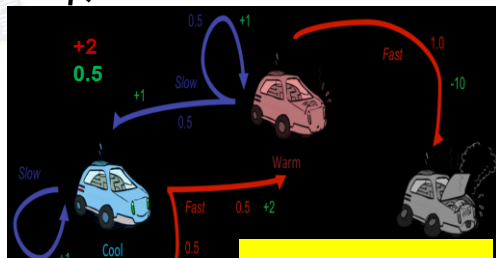
$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

- Below is done only for state : "Cool". Repeat this for all the states "Warm", "Overheated" per iteration

$$V(\text{cool}) = \max \begin{cases} \text{slow: } 2.8 \\ \text{fast: } 3.35 \end{cases}$$

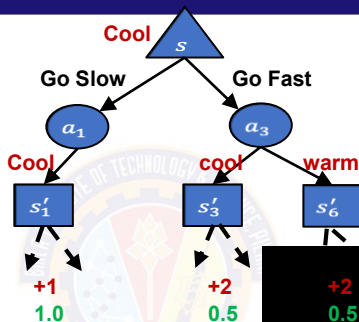
$$V(\text{cool}) = \max \begin{cases} 1 * [1 + 0.9(2)] \\ (0.5 * [2 + 0.9(2)]) + (0.5[2 + 0.9(1)]) \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} P(c|c,s)[r(c,s,c) + 0.9(v(c))] \\ (P(c|c,f)[r(c,f,c) + 0.9(v(c))]) + (P(w|c,f)[r(c,f,w) + 0.9(v(w))]) \end{cases}$$



Dynamic Programming – Value Iteration

			
V_0	0	0	0
V_1	2	1	0
V_2	3.35	2.35	0
.....			
V_{49}	~15.41	~14.41	0



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}$)

Repeat $\epsilon = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

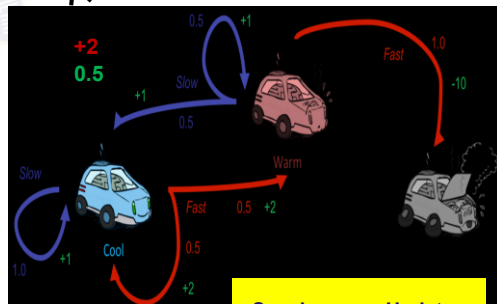
$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that

$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

- $\Delta = \sim 0.009$ less than threshold $\epsilon = 0.01$
- Algorithm converges slowly. Without ϵ converges $\sim 208^{\text{th}}$
- Can faster convergence possible ? Yes via Asynchronous Value Updates.** Idea is to use the recent values $V(S')$ even within the same iteration !
- One iteration is shown in next slide



Synchronous Updates

Dynamic Programming – Value Iteration

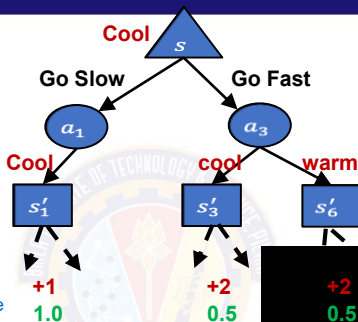
			
V_0	0	0	0
V_1	2		
V_2			

Below is done only for state : "Cool". In this same iteration while updating the $V(\text{Warm})$ use recent value of $V(\text{Cool}) = 2$ not 0.

$$V(\text{cool}) = \max \begin{cases} \text{slow: } 1 \\ \text{fast: } 2 \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} 1 * [1 + 0.9(0)] \\ (0.5 * [2 + 0.9(0)]) + (0.5[2 + 0.9(0)]) \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} P(c|c,s)[r(c,s,c) + 0.9(v(c))] \\ (P(c|c,f)[r(c,f,c) + 0.9(v(c))]) + (P(w|c,f)[r(c,f,w) + 0.9(v(w))]) \end{cases}$$



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}$)

Repeat $\theta = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

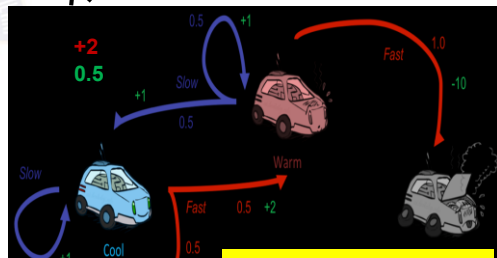
$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that

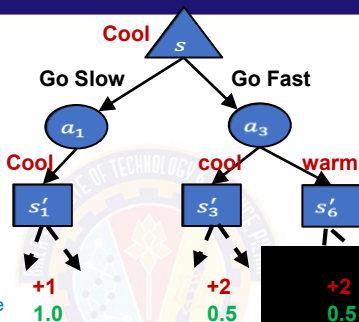
$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$



Asynchronous Updates

Dynamic Programming – Value Iteration

			
V_0	0	0	0
V_1	2	1.9	0
V_2			



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}$)

Repeat $\theta = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that

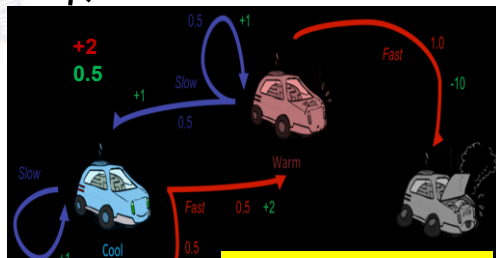
$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

- Below is done only for state : "Cool". In this same iteration while updating the $V(\text{Warm})$ use recent value of $V(\text{Cool}) = 2$.

$$V(\text{warm}) = \max \begin{cases} \text{fast: } -10 \\ \text{slow: } 1.9 \end{cases}$$

$$V(\text{warm}) = \max \begin{cases} 1 * [-10 + 0.9(0)] \\ (0.5 * [1 + 0.9(2)]) + (0.5[1 + 0.9(0)]) \end{cases}$$

$$V(\text{warm}) = \max \begin{cases} P(o|w,f)[r(w,f,o) + 0.9(v(o))] \\ (P(c|w,s)[r(w,s,c) + 0.9(v(c))] + (P(w|w,s)[r(w,s,w) + 0.9(v(w))]) \end{cases}$$

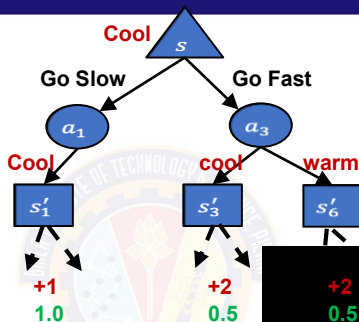


Asynchronous Updates

Dynamic Programming – Value Iteration

			
V_0	0	0	0
V_1	2	1.9	0
V_2	3.75	3.54	0
.....			
V_{40}	~15.44	~14.44	0

- $\Delta = \sim 0.008$ less than threshold $\epsilon = 0.01$
- Algorithm converges fast. Without ϵ converges at $\sim 157^{\text{th}}$ iteration!



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}$)

Repeat $\epsilon = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

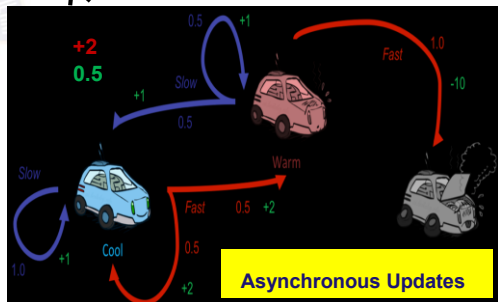
$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

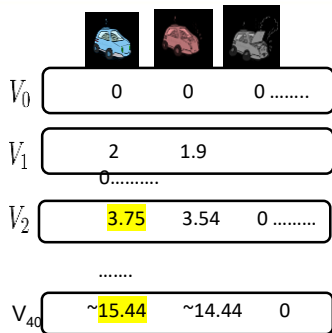
until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that

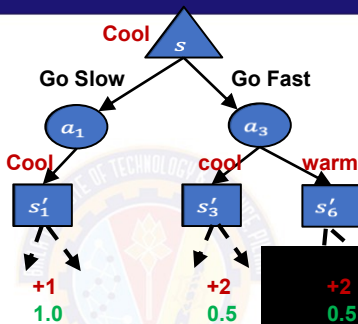
$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$



Dynamic Programming – Value Iteration



State	Best Action
Cool	Fast
Warm	Slow
Overheated	-



Value Iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}$)

Repeat $\epsilon = 0.01$

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

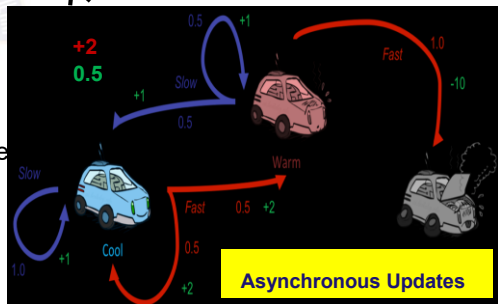
$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, $\pi \approx \pi_*$, such that $\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

- After convergence extract the policy by choosing the action that lead to the converged maximum value here from V_{40} th iteration



Asynchronous Updates

Dynamic Programming Solution

In many cases, sometimes policies is observed to converge faster than value!

So why wait for entire value updates?



Policy Iteration Algorithm

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_*,$$

Dynamic Programming

1 Given

- $P(s' | s, a)$, $R(s, a, s')$, $P(s', r | s, a)$

2 To find

- Policy π

3 Solution

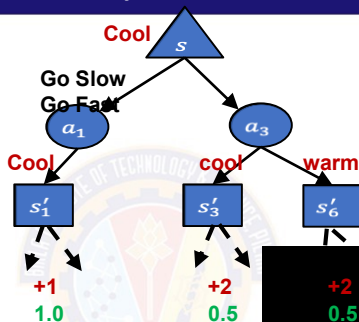
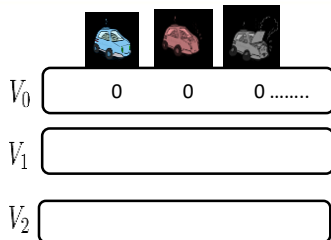
- Policy Iteration (PI): Prefer this in a relatively smaller state space where intermediate termination provides explicit interpretable policy
- Value Iteration (VI): Prefer this in a relatively larger state space where fast convergence to optimal policy is obtained at the end

4 Idea

- PI : One Step Look ahead ☐ Evaluate ☐ Improve ☐ *Number of state sweeps (updates) can be controlled if required (similar to mini-batch /stochastic processing). Evaluate & Improve steps are alternatively are done .*
- VI : One Step Look ahead ☐ Control

VI can be thought of truncated PI where evaluation is done fully before improvement.

Dynamic Programming – Policy Iteration



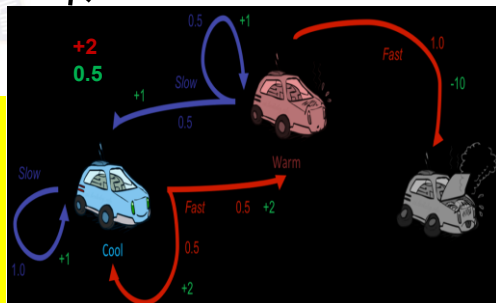
Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$.

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
 $\theta = 0.01$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

π_0

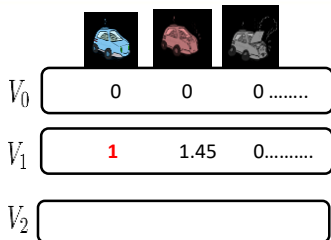
State	Best Action
Cool	Slow
Warm	Slow
Overheated	-

We will use asynchronous for updates in this example. Note that even though above backup diagram depicts all possible transition, this algorithm computes value from only one of the action as given by the initialized policy at the start of every iteration!!!



Policy Evaluation Starts

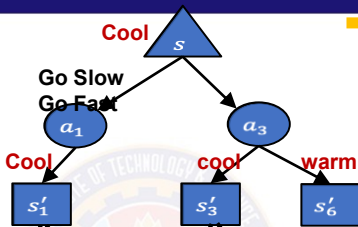
– Policy Iteration



- The value uses the initialized policy to select only one action GREEDILY. Hence below cancelled equations differentiated PI vs VI
- $$V(\text{cool}) = \max \begin{cases} \text{slow: } 1 \\ \text{fast: } 2 \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} 1 * [1 + 0.9(0)] \\ (0.5 * [2 + 0.9(0)]) + (0.5[2 + 0.9(0)]) \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} P(c|c,s)[r(c,s,c) + 0.9(v(c))] \\ (P(c|c,f)[r(c,f,c) + 0.9(v(c))]) + (P(w|c,f)[r(c,f,w) + 0.9(v(w))]) \end{cases}$$

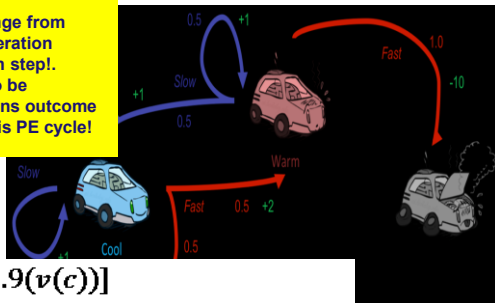


This slide show what change from Value iteration to Policy iteration algorithm in the Evaluation step!. Since only action needs to be assessed, MAX of all actions outcome is no longer required in this PE cycle!

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$.

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

$\theta = 0.01$

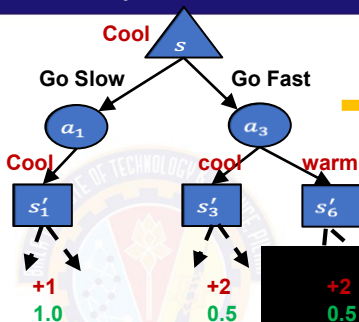


Policy Evaluation Completes

– Policy Iteration

			
V_0	0	0	0
V_1	1	1.45	0
V_2	1.9	2.51	0
V_{225}	~10	~10	0

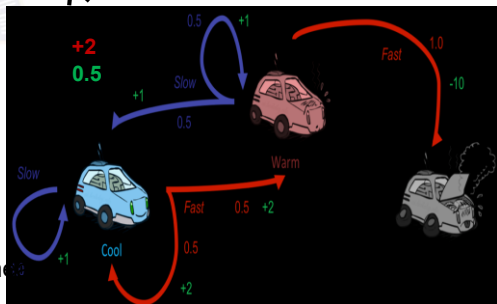
- In PE, either synchronous or asynchronous value updates can be followed
- If Threshold of 0.01 is applied then approx., at 46th iteration algorithm converges
- Assume** that Asynchronous method is used & Thresholding is not applied. In this case at approx., at 225th iteration values converge



Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$.

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

$\theta = 0.01$



Policy Improvement

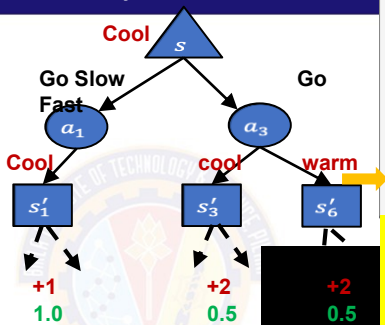
– Policy Iteration

			
V_0	0	0	0
V_1	1	1.45	0
V_2	1.9	2.51	0
V_{255}	~10	~10	0

$$V(\text{cool}) = \max \begin{cases} \text{slow: } 10 \\ \text{fast: } 11 \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} 1 * [1 + 0.9(\text{10})] \\ (0.5 * [2 + 0.9(\text{10})]) + (0.5[2 + 0.9(\text{10})]) \end{cases}$$

$$V(\text{cool}) = \max \begin{cases} P(c|c,s)[r(c,s,c) + 0.9(v(c))] \\ (P(c|c,f)[r(c,f,c) + 0.9(v(c))] + (P(w|c,f)[r(c,f,w) + 0.9(v(w))]) \end{cases}$$

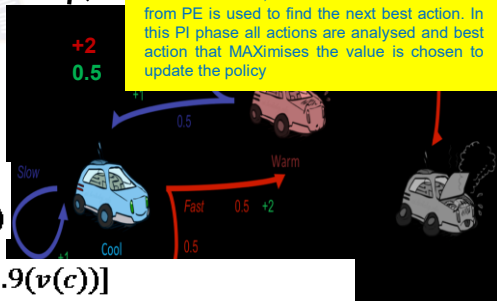


Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$.

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s))[r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

$\theta = 0.01$

After PE is done, PI starts where the last value from PE is used to find the next best action. In this PI phase all actions are analysed and best action that MAXimises the value is chosen to update the policy



Policy Improvement

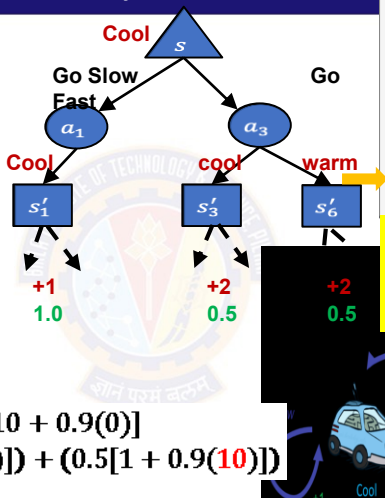
– Policy Iteration

			
V_0	0	0	0
V_1	1	1.45	0
V_2	1.9	2.51	0
V_{255}	~10	~10	0

$$V(\text{warm}) = \max \begin{cases} \text{fast: } -10 \\ \text{slow: } 10 \end{cases}$$

$$V(\text{warm}) = \max \begin{cases} 1 * [-10 + 0.9(0)] \\ (0.5 * [1 + 0.9(\text{red})]) + (0.5[1 + 0.9(\text{red})]) \end{cases}$$

$$V(\text{warm}) = \max \begin{cases} P(o|w,f)[r(w,f,o) + 0.9(v(o))] \\ (P(c|w,s)[r(w,s,c) + 0.9(v(c))] + (P(w|w,s)[r(w,s,w) + 0.9(v(w))]) \end{cases}$$

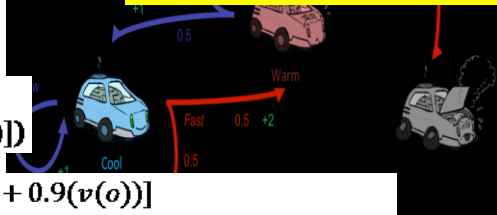


Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$.

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s))[r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

$\theta = 0.01$

After PE is done, PI starts where the last value from PE is used to find the next best action. In this PI phase all actions are analysed and best action that MAXimises the value is chosen to update the policy



Policy Improvement Completes

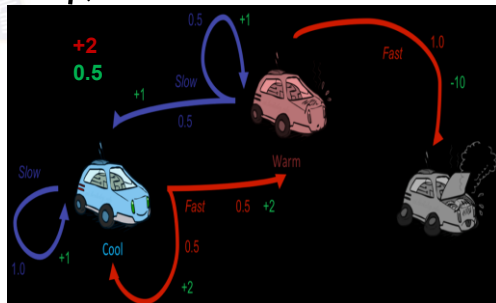
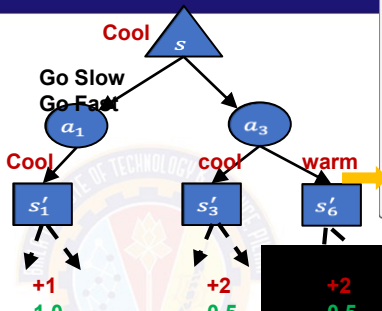
– Policy Iteration

			
V_0	0	0	0
V_1	1	1.45	0
V_2	1.9	2.51	0
V_{225}	~10	~10	0

$$V(warm) = \max_{\pi_1} \begin{cases} \text{fast: } -10 \\ \text{slow: } 10 \end{cases}$$

State	Best Action
Cool	Fast
Warm	Slow
Overheated	-

$$V(cool) = \max \begin{cases} \text{slow: } 10 \\ \text{fast: } 11 \end{cases}$$

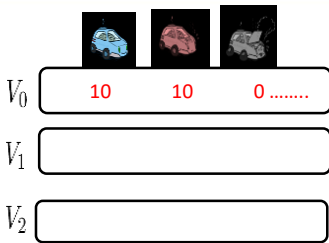


Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$.

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
 $\theta = 0.01$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Continue another cycle of – Policy Iteration

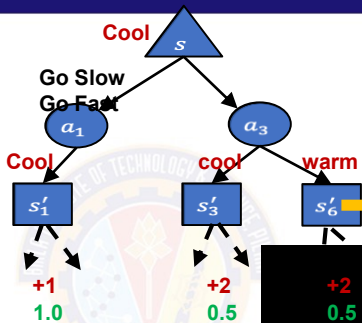
PE & PI



π_1

State	Best Action
Cool	Fast
Warm	Slow
Overheated	-

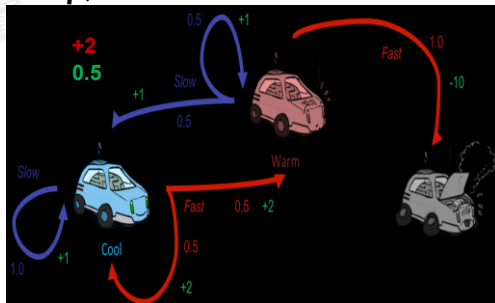
After this cycle of algorithm converges with values :
 $V(\text{cool}) = 15.5$
 $V(\text{Warm}) = 14.5$



Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$.

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

$\theta = 0.01$



Dynamic Programming - Policy Iteration

1 Observation

- Policy iteration consists of two simultaneous, interacting processes, one making the value function consistent with the current policy (policy evaluation), and the other making the policy greedy with respect to the current value function (policy improvement).
- In policy iteration, these two processes alternate, each completing before the other begins, **but this is not really necessary**.

Solution : GPI Generalized Policy Iteration

- Generalized policy iteration (GPI) to refer to the general idea of letting policy-evaluation and policy improvement processes interact, independent of the granularity and other details of the two processes. Almost all reinforcement learning methods are well described as GPI.

Idea (Do this as an exercise for given problem)

In one cycle perform only one iteration of Policy Evaluation (ie., Value Updates) without waiting for convergence immediately follow by Policy Improvement.

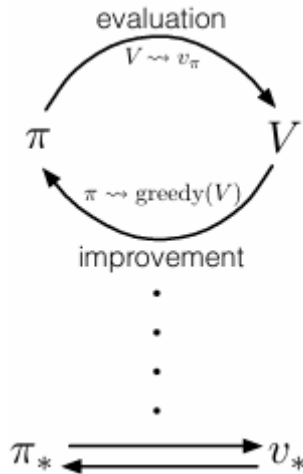


TABLE OF CONTENTS

- 1 [MDP](#)
- 2 [DP](#)
- 3 [MC \(to be updated on or after session 6\)](#)
- 4 [TD \(to be updated on or after session 7\)](#)

